Cryptography Engineering

- Lecture 1 (Oct 21, 2026)

- Today:
  - Symmetric Primitives

Hash Function

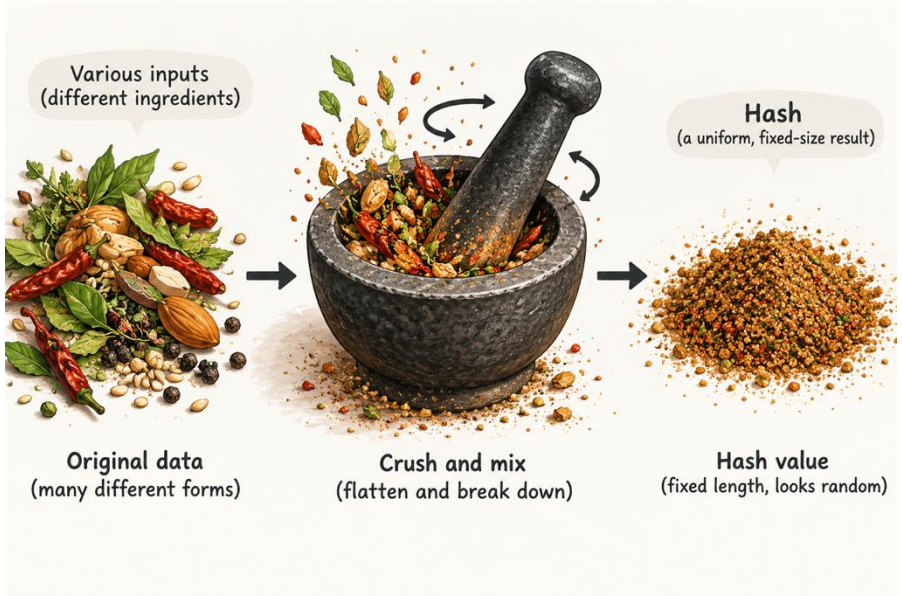
- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string **(digest)**

Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string **(digest)**



Generated by ChatGPT-6 Sol

Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string

  - Security: collision resistance, (second) preimage resistance, …
  - SHA (Secure Hash Algorithm) family: ~~SHA-1~~, SHA-2, SHA-3

Security Properties of Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string

  - Security: collision resistance, (second) preimage resistance, …
  - SHA (Secure Hash Algorithm) family: ~~SHA-1~~, SHA-2, SHA-3

- Find collision: Find x, x' s.t. $x \neq x'$ and $H(x) = H(x')$
- **Collision resistance**: Finding any collision is hard (i.e., no polynomial-time algorithm)

Security Properties of Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string

  - Security: collision resistance, (second) preimage resistance, …
  - SHA (Secure Hash Algorithm) family: ~~SHA-1~~, SHA-2, SHA-3

- **Preimage resistance**: Given an arbitrary bit string s, it is hard to find x s.t. $h(x) = s$

Security Properties of Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string

  - Security: collision resistance, (second) preimage resistance, …
  - SHA (Secure Hash Algorithm) family: ~~SHA-1~~, SHA-2, SHA-3

- **Second Preimage resistance**: Given x, it is hard to find $x' \neq x$ s.t. $h(x) = h(x')$

Security Properties of Hash Function

- Hash function:

  H(“…arbitrary-length string…”) = a fixed length (e.g., 256) bit string

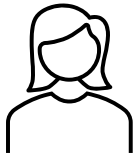
  - Security: collision resistance, (second) preimage resistance, …
  - SHA (Secure Hash Algorithm) family: ~~SHA-1~~, SHA-2, SHA-3

- Quick questions: Let h be a hash function and *test.rs* be a file
  - Given h(*test.rs*) (compute the hash of the whole file), can we recover *test.rs*?
  - Can we find *another* file such that its hash is h(*test.rs*)?
  - Can we find two arbitrary different files such that their hashes are the same?

Symmetric Primitives - SKE

- Symmetric-key Encryption

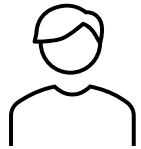
Alice





key

Bob





key

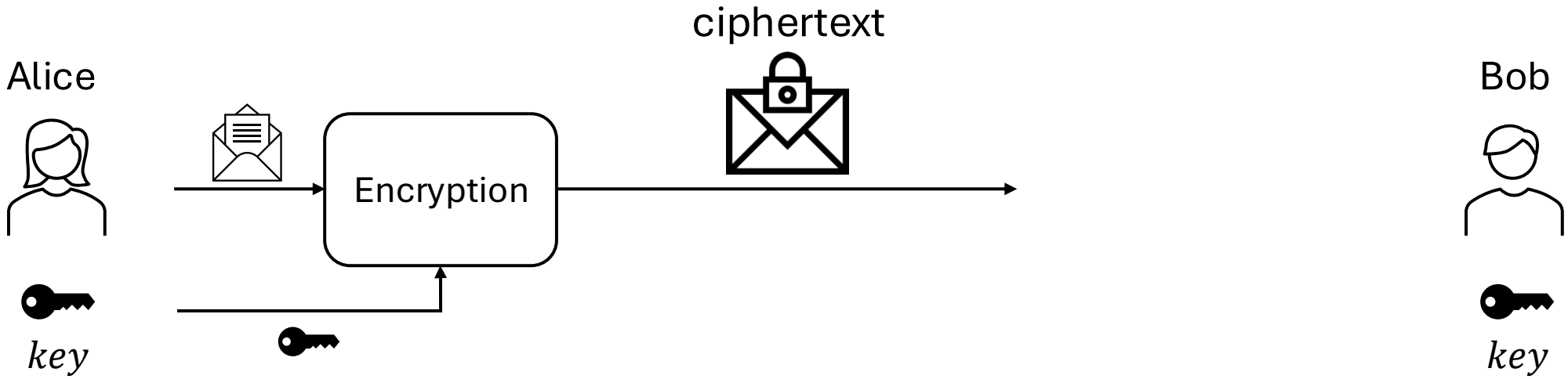
Symmetric Primitives - SKE

- Symmetric-key Encryption



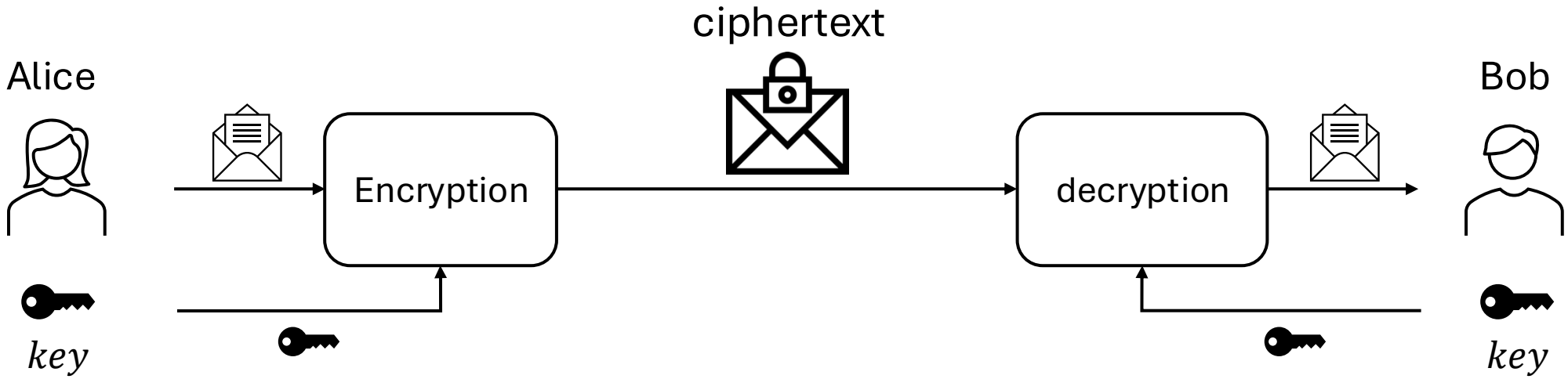
Symmetric Primitives - SKE

- Symmetric-key Encryption



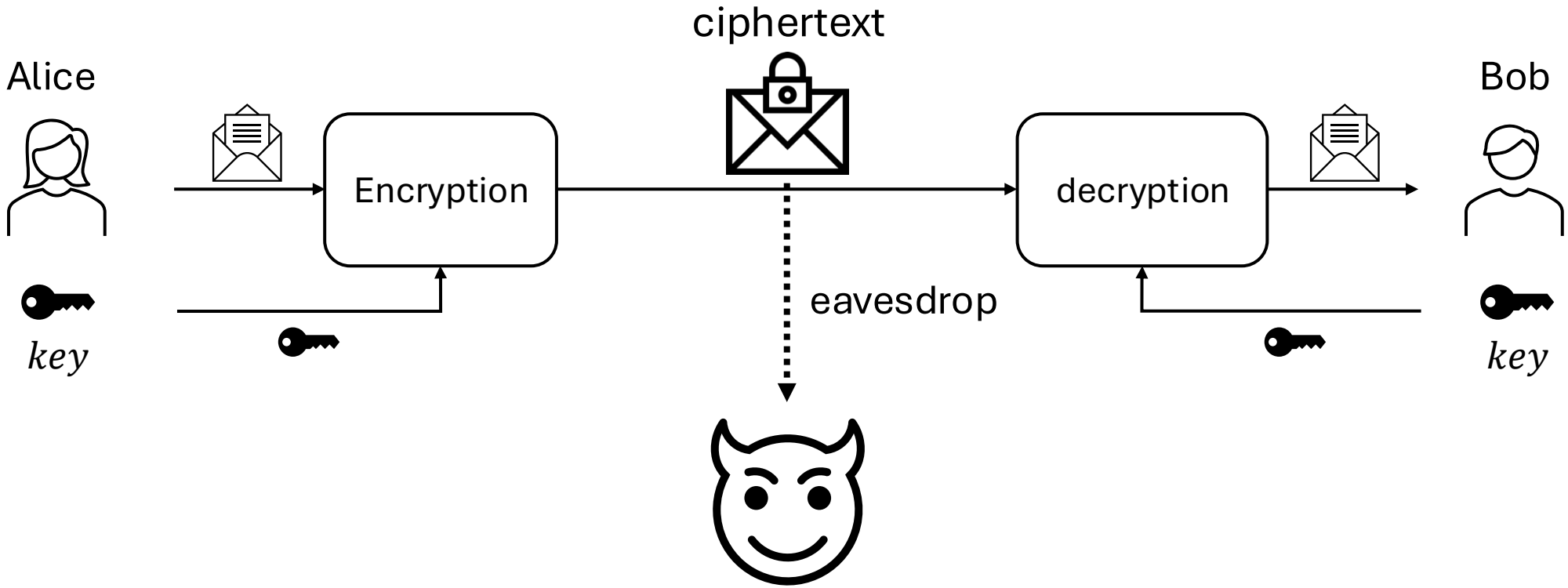
Symmetric Primitives - SKE

- Symmetric-key Encryption



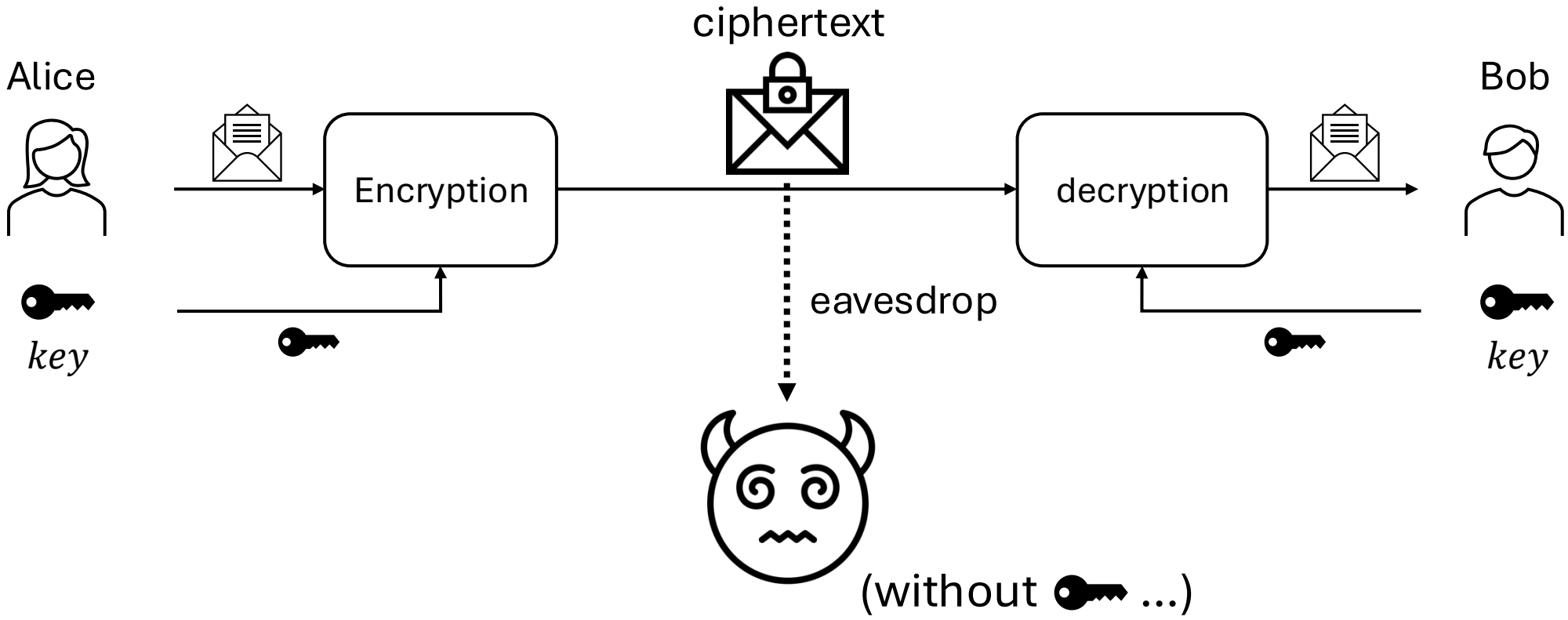
Symmetric Primitives - SKE

- Symmetric-key Encryption



Symmetric Primitives - SKE

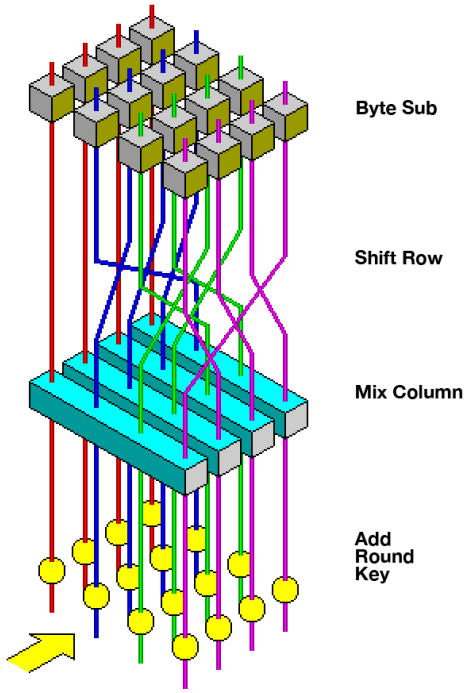
- Symmetric-key Encryption (**Confidentiality**)



Symmetric Primitives - SKE

- Symmetric-key Encryption
  - AES (Advanced Encryption Standard)
  - Fixed-length encryption (block cipher)

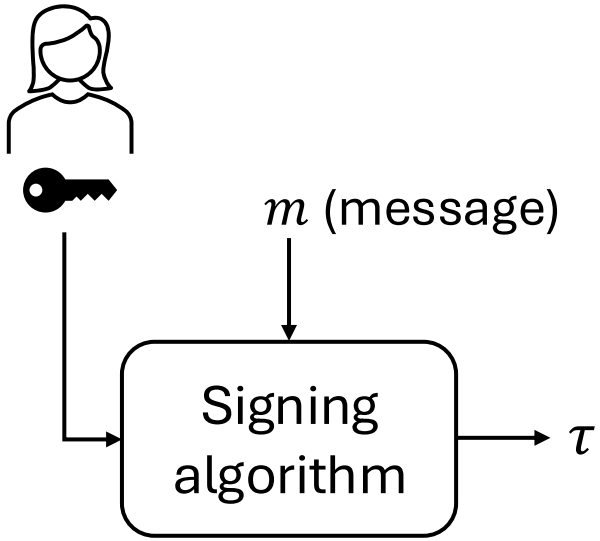
- Extend to arbitrary-length encryption **via Mode of Operation**
  - CBC, CTR, …

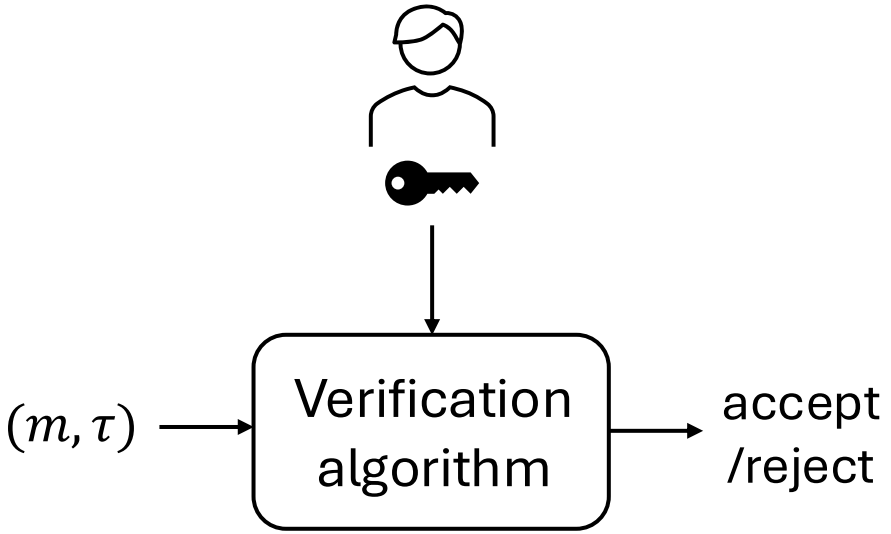


(Image from Wikipedia)

Symmetric Primitives - MAC

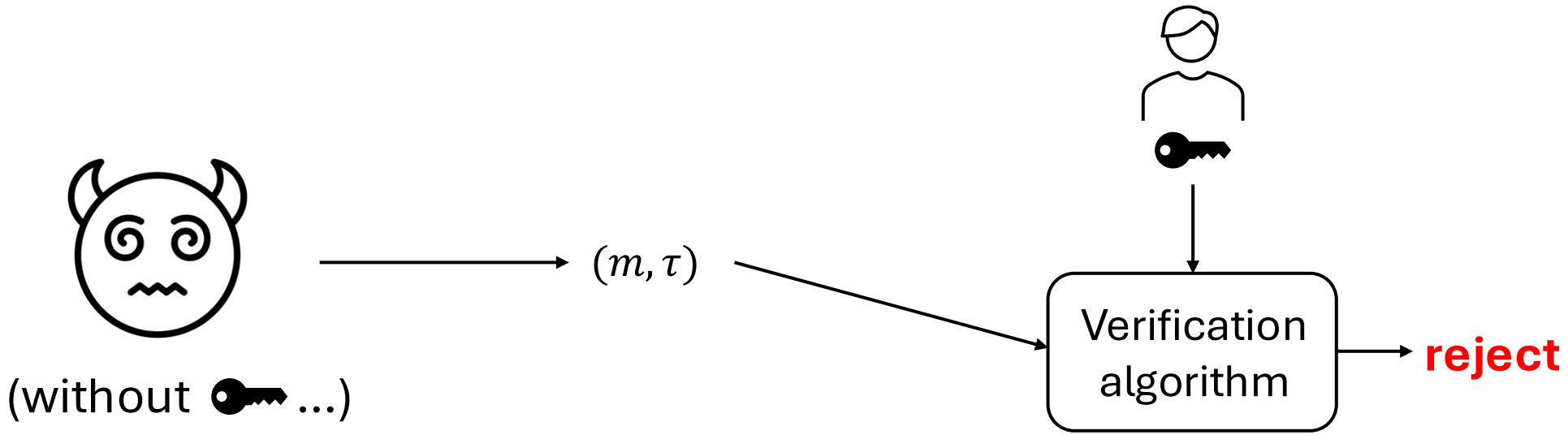
- Message Authentication Code (MAC)
  - **Integrity** (…cannot forge a valid MAC tag without knowing the secret key…)
  - **HMAC** (Hash-based message authentication code)





Symmetric Primitives - MAC

- Message Authentication Code (MAC)
  - **Integrity** (…cannot forge a valid MAC tag without knowing the secret key…)
  - **HMAC** (Hash-based message authentication code)



Symmetric Primitives - KDF

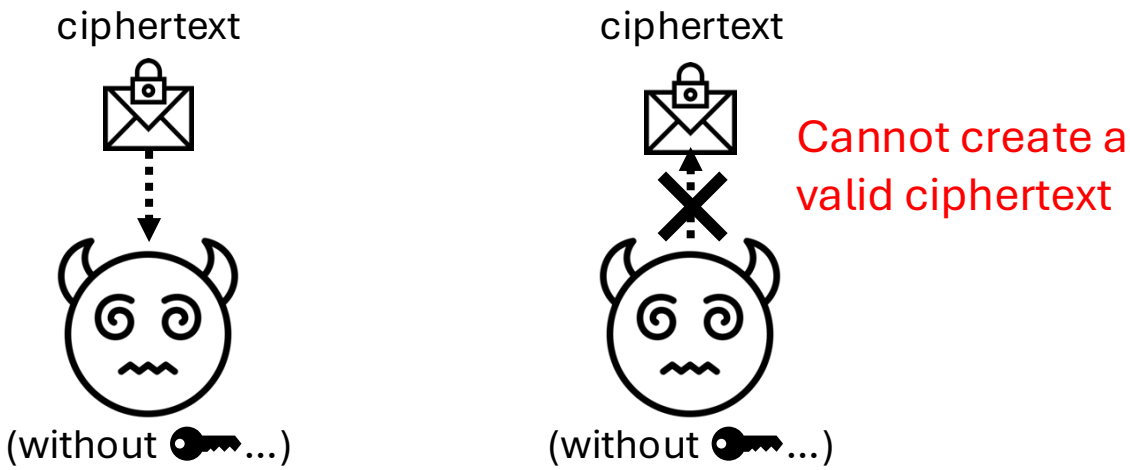
- Key Derivation Function (KDF)

  KDF(“…shared secret with randomness…”) = a symmetric key

  - Used to derive a key for symmetric key encryption
  - HKDF (based on HMAC)
  - Derive keys of arbitrary lengths

Symmetric Primitives - AE

- Authenticated Encryption
  - Symmetric Encryption
  - Not only **Confidentiality**, but also **Integrity**



Symmetric Primitives - AE

- Authenticated Encryption
  - Symmetric Encryption
  - Not only **Confidentiality**, but also **Integrity**

- Approaches to authenticated encryption:
  - Encrypt-then-MAC (EtM), …

- Authenticated Encryption **with Associated Data (AEAD)**
  - Ensure the message **and additional data (like headers)** are authenticated and encrypted securely.
  - AES-GCM, ChaCha20-Poly1305, …

Next Topic

- More discussions on hash functions and AE(AD).

Exercise

- Practice your Rust skill

1. **Hash functions:**
   (1) Input a file; (2) Compute its digital fingerprint (using SHA3-256); (3) Verify the digital fingerprint.
2. **AES:**
   (1) Generate a random key; (2) Encrypt an arbitrary string using AES; (3) Decrypt the ciphertext. (4) Modify the ciphertext and decrypt it (using the same key) again.
3. **HMAC:**
   (1) Generate a random key; (2) Generate a MAC code for an arbitrary string using HMAC; (3) Verify it.
   (4) Modify the MAC code and verify it again with the original string.
4. **AEAD:**
   (1) Generate a random key; (2) Generate an arbitrary string using AES-GCM; (3) Decrypt it;
   (4) Modify the ciphertext and decrypt it again; (5) Modify the AD (associate date) and decrypt again.
5. **KDF:** Using HKDF to derive a key (using arbitrary inputs).

Exercise

- Practice your understanding about cryptography

1. What are the relations between **Collision Resistance**, **Preimage resistance, and Second Preimage resistance?**

2. Here is the construction of HMAC:

$$\text{HMAC}(K, m) = H\left((K' \oplus opad) \parallel H\big((K' \oplus ipad) \parallel m\big)\right)$$

where H is the underlying hash function, opad and ipad are some fixed bit strings, and

$$K' = \begin{cases} K \\ H(K) \end{cases} \quad \text{if } K \text{ is larger than the output length of } H.$$

If you modify m, then what property of H prevents the original HMAC tag from remaining valid (under this modified m?

Further Reading

- AEAD and AES-GCM: https://en.wikipedia.org/wiki/Galois/Counter_Mode
- HKDF: https://en.wikipedia.org/wiki/HKDF
- Mode of operation: https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation
- Awesome Cryptography Rust: https://github.com/rust-cc/awesome-cryptography-rust